

Distributed Systems

Lecture24: Arbitrary Failure

Mohammad Reza Shakournia

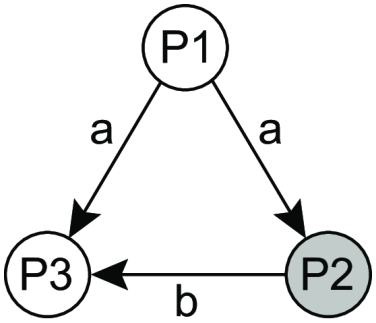
shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

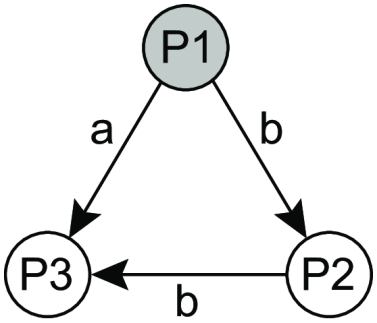
Consensus Under Arbitrary Failure Semantics

- **Essence**

  We consider process groups in which communication between process is inconsistent.



Improper forwarding

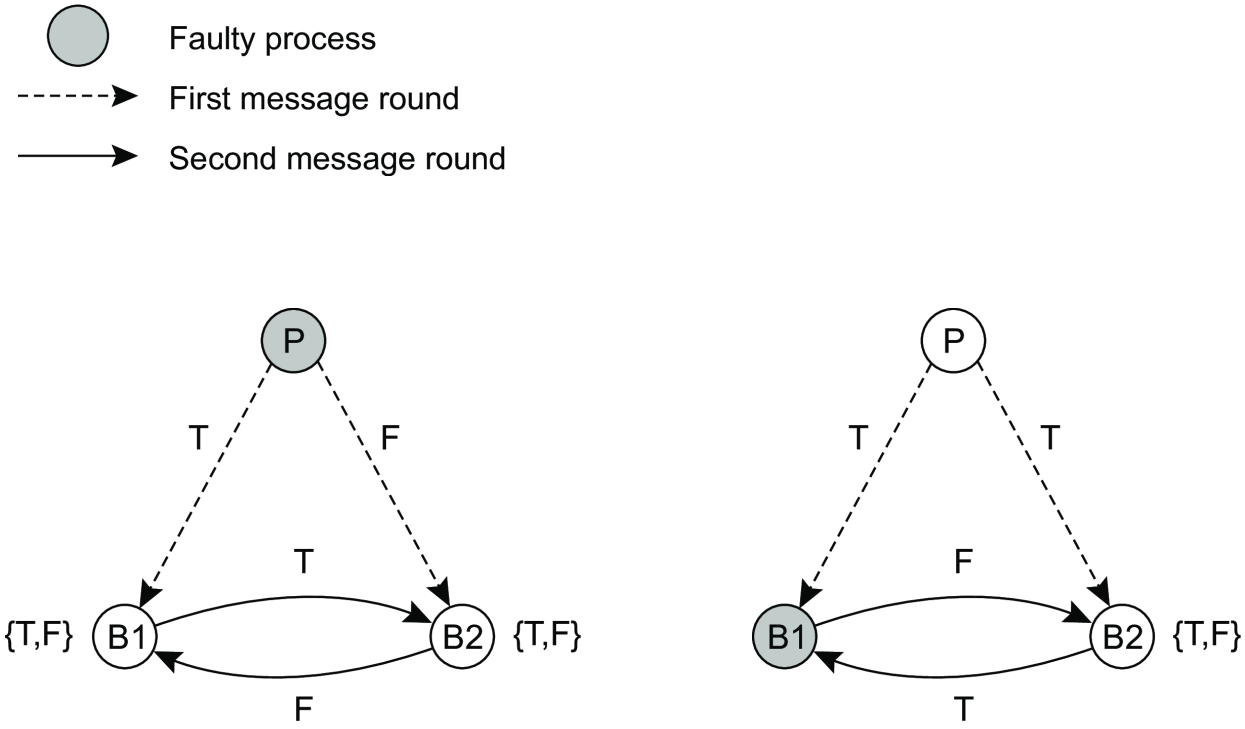


Different messages

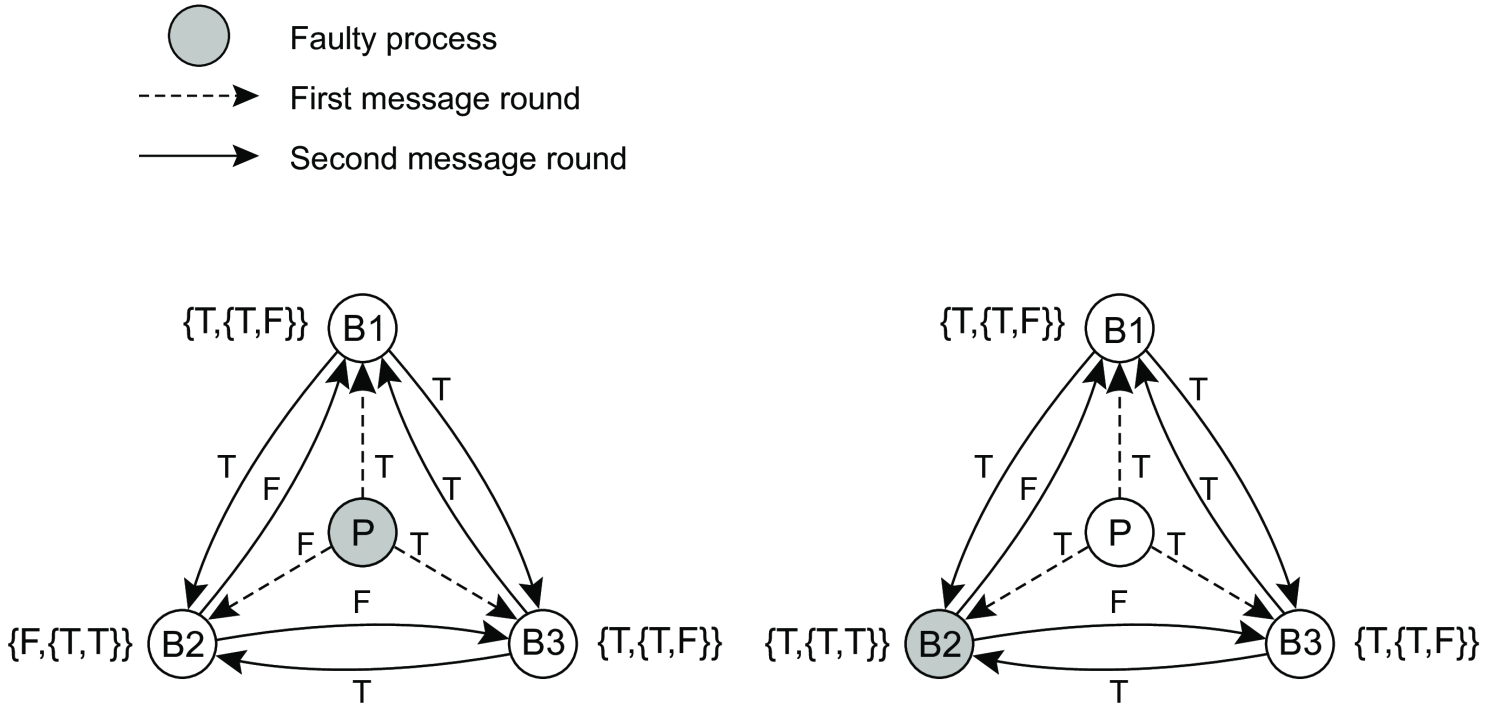
Consensus Under Arbitrary Failure Semantics

- **System model**
  - We consider a **primary** P and $n-1$ backups $B_1, \ldots, B_{n-1}$.
  - A client sends $v \in \{T, F\}$ to P.
  - Messages may be **lost**, but this can be detected.
  - Messages **cannot be corrupted beyond detection**.
  - A receiver of a message can **reliably detect its sender**.
- **Byzantine agreement: requirements**
  - BA1: Every nonfaulty backup process stores the same value.
  - BA2: If the primary is nonfaulty then every nonfaulty backup process stores exactly what the primary had sent.
- **Observation**
  - Primary faulty ⇒ BA1 says that backups may store the same, but different (and thus wrong) value than originally sent by the client.
  - Primary not faulty ⇒ satisfying BA2 implies that BA1 is satisfied.

Why Having 3K Processes is Not Enough



Why Having 3K+1 Processes is Enough



Practical Byzantine Fault Tolerance (PBFT)

- **Background**

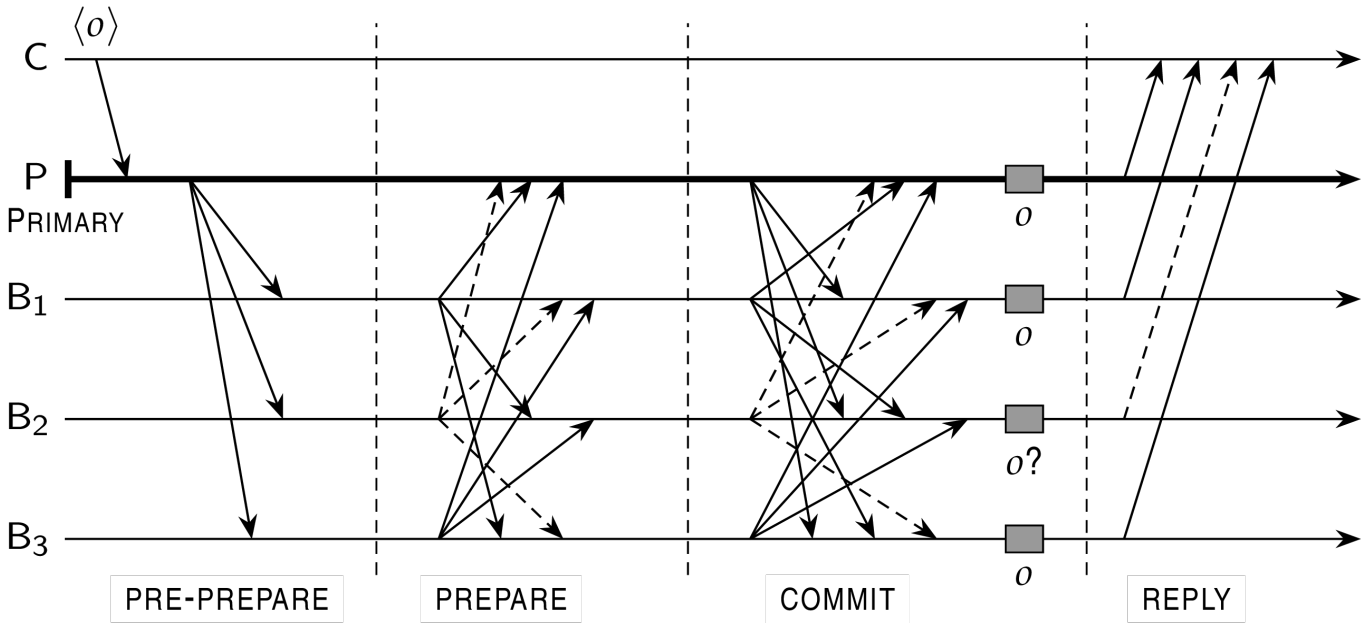
  One of the first solutions that managed to achieve Byzantine fault tolerance while keeping performance acceptable. Popularity has increased with the introduction of permissioned blockchains.

- **Assumptions**
  - A server may exhibit arbitrary failures
  - Messages may be lost, delayed, and received out of order
  - Messages have an identifiable sender (i.e., they are signed)
  - Partially synchronous execution model

- **Essence**

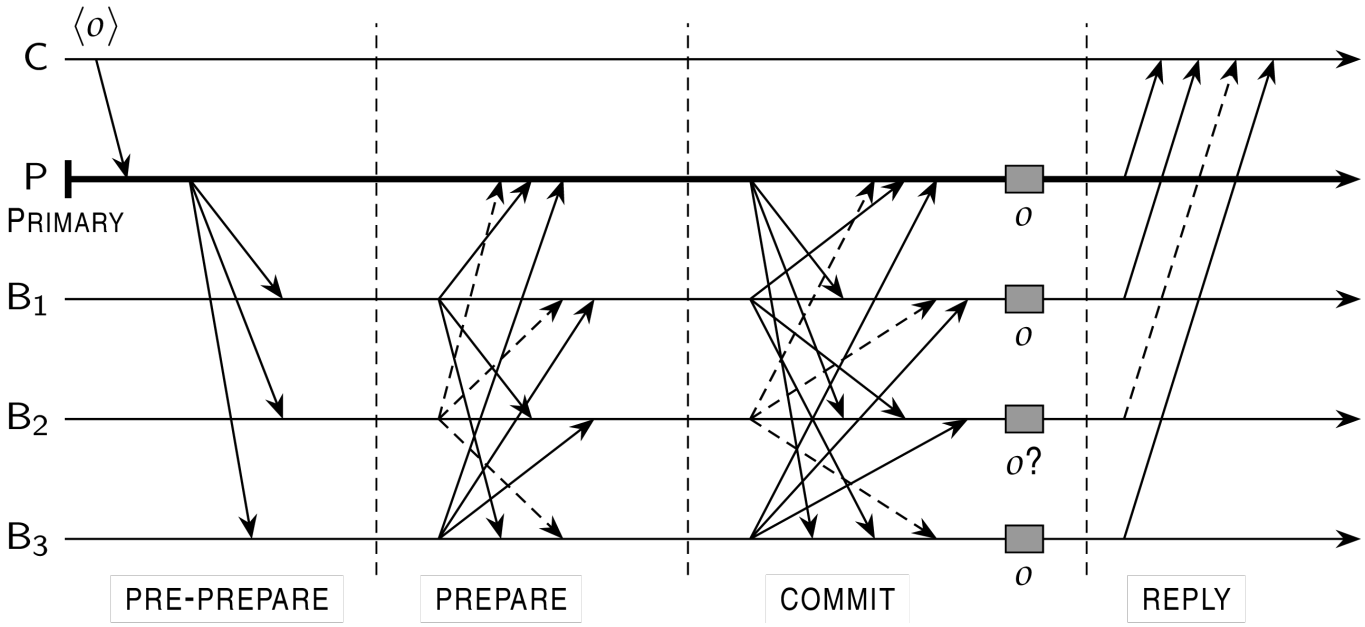
  A primary-backup approach with $3k + 1$ replica servers.

PBFT: Four Phases



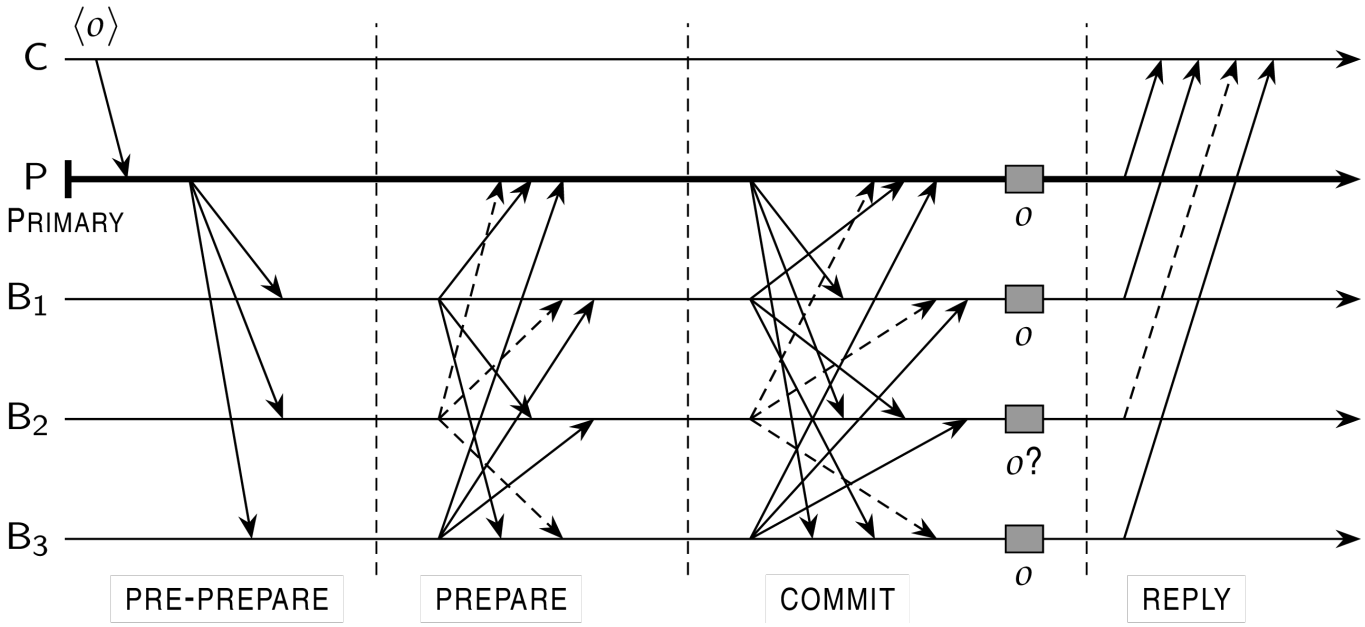
- C is the client
- P is the primary
- B_1, B_2, B_3 are backups
- Assume B_2 is faulty

PBFT: Four Phases



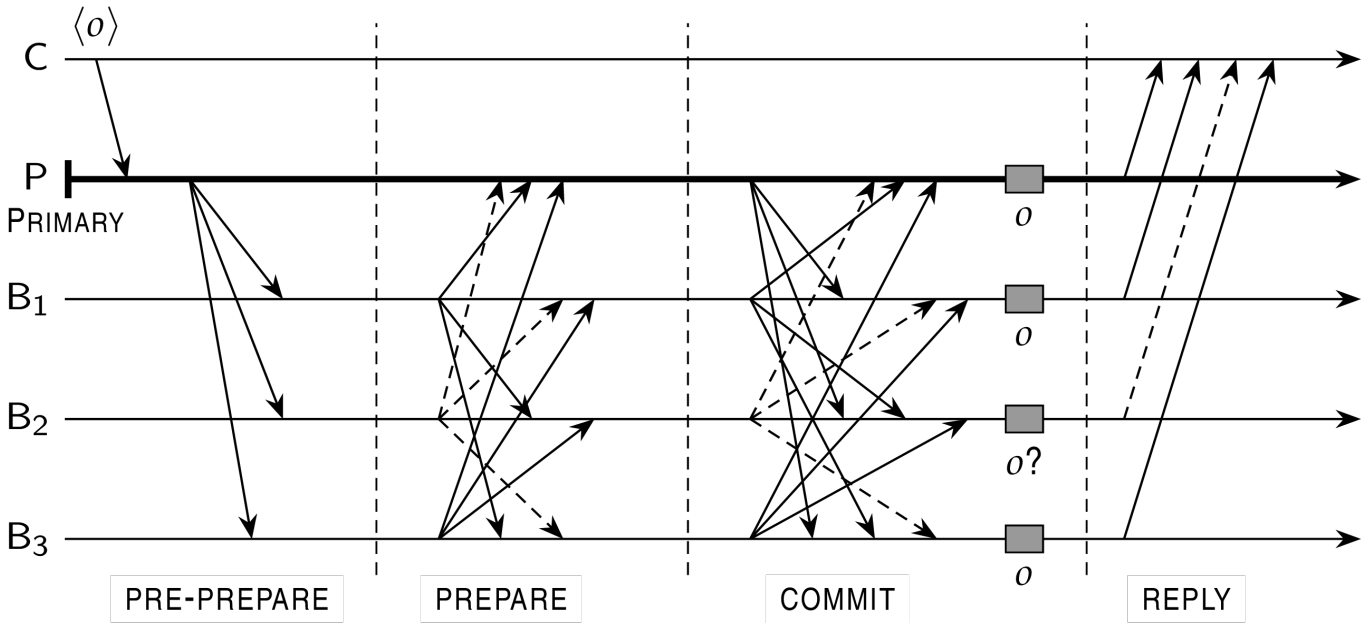
- All servers assume to be working in a current view v.
- C requests operation o to be executed
- P timestamps o and sends PRE-PREPARE(t, v, o)
- Backup B_i accepts the pre-prepare message if it is also in v and has not accepted an operation with timestamp t before.

PBFT: Four Phases



- B_i broadcasts PREPARE(t, v, o) to all (including the primary)
- Note: a nonfaulty server will eventually log $2k$ messages PREPARE(t, v, o) (including its own) ⇒ consensus on the ordering of o.
- Note: it doesn't matter what faulty B_2 sends, it cannot affect joint decisions by P, B_1, B_3.

PBFT: Four Phases



- All servers broadcast COMMIT(t, v, o)
- The commit is needed to also make sure that o can be executed now, that is, in the current view v.
- When $2k$ messages have been collected, excluding its own, the server can safely execute o and reply to the client.

PBFT: When The Primary Fails

- **Issue**

  When a backup detects the primary failed, it will broadcast a **view change** to view $v + 1$. We need to ensure that any **outstanding request** is executed once and only once by all nonfaulty servers. The operation needs to be handed over to the new view.

- **Procedure**
  - The next primary P^* is known deterministically
  - A backup server broadcasts VIEW-CHANGE$((\ v\ +\ 1, P\ ))$: P is the set of prepares it had sent out.
  - P^* waits for $2k + 1$ view-change messages, with $X = \bigcup P$ containing all previously sent prepares.
  - P^* sends out NEW-VIEW$((\ v\ +\ 1, X, O\ ))$ with O a new set of pre-prepare messages.

- **Essence**: this allows the nonfaulty backups to replay what has gone on in the previous view, if necessary, and bring o into the new view $v + 1$.

Realizing Fault Tolerance

- **Observation**

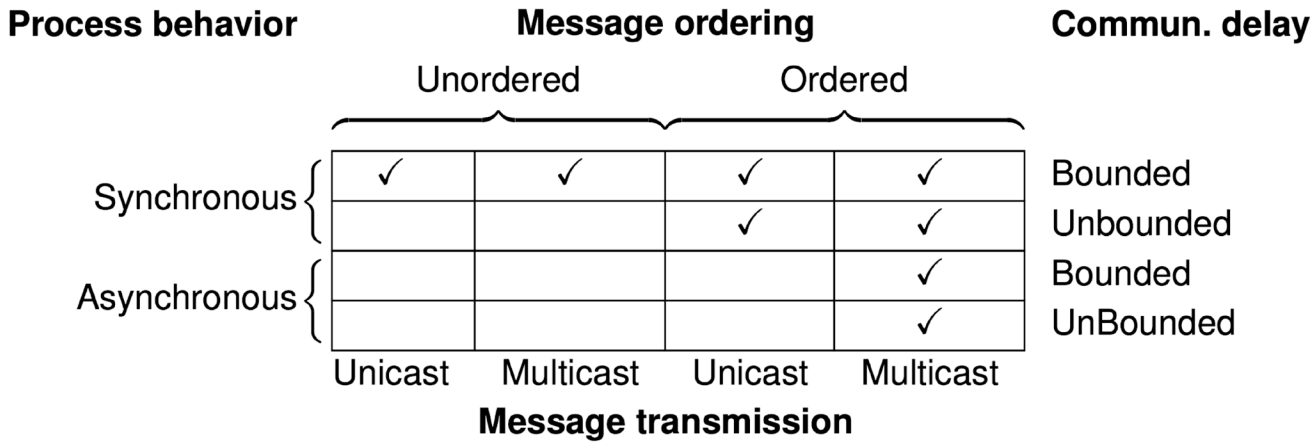
  Considering that the members in a fault-tolerant process group are so tightly coupled, we may bump into considerable performance problems, but perhaps even situations in which realizing fault tolerance is impossible.

- **Question**

  Are there limitations to what can be readily achieved?

  - What is needed to enable reaching consensus?
  - What happens when groups are partitioned?

Distributed Consensus: When Can It Be Reached



- **Formal requirements for consensus**
  - Processes produce the same output value
  - Every output value must be valid
  - Every process must eventually provide output

Consistency, Availability, Partitioning

- **CAP theorem**

  Any networked system providing shared data can provide only two of the following three properties:

  - **C:** consistency, by which a shared and replicated data item appears as a single, up-to-date copy
  - **A:** availability, by which updates will always be eventually executed
  - **P:** Tolerant to the partitioning of process group.

- **Conclusion**

  In a network subject to communication failures, it is impossible to realize an atomic read/write shared memory that guarantees a response to every request.

CAP Theorem Intuition

- **Simple Situation: Two Interacting Processes**
  - P and Q can no longer communicate:
    - Allow P and Q to go ahead ⇒ no consistency
    - Allow only one of P, Q to go ahead ⇒ no availability
  - P and Q have to be assumed to continue communication ⇒ no partitioning allowed.
- **Fundamental Question**

  What are the practical ramifications of the CAP theorem?

Failure Detection

- **Issue**

  How can we reliably detect that a process has actually crashed?

- **General model**
  - Each process is equipped with a failure detection module
  - A process P probes another process Q for a reaction
  - If Q reacts: Q is considered to be alive (by P)
  - If Q does not react within t time units: Q is suspected to have crashed

- **Observation for a synchronous system**

$$a \textit{ suspected crash} \equiv \textit{a known crash}$$

Practical Failure Detection

- **Implementation**
  - If P did not receive **heartbeat** from Q within time t: P suspects Q.
  - If Q later sends a message (which is received by P):
    - P stops suspecting Q
    - P increases the timeout value t
  - Note: if Q did crash, P will keep suspecting Q.